AlphaPaaS Container & Kubernetes Security Standard

Reference: CKS · Version 4.1 · Effective 1 February 2026

Document owner	AlphaInsure Group Security — Cloud Security
Approved by	AlphaInsure Chief Information Security Officer
Review cycle	Annual
Previous version	4.0 (November 2024)
Classification	Internal
Regulatory basis	MTSB for OpenShift; CIS Benchmark for OpenShift 4.x
Related standards	AHS, ARS, ASC, DCH

1. Purpose and scope

This standard defines security requirements for container images, Kubernetes workloads, and cluster-level configuration on AlphaPaaS. Many clauses derive from the Group Security Minimal Technical Security Baseline (MTSB), which in turn mandates the CIS OpenShift benchmark. Where a clause maps to a specific CIS control, the CIS identifier is given in parentheses.

Compliance is mandatory for all workloads on AlphaPaaS. Waivers require sign-off from Group Security and are granted only in exceptional circumstances.

2. Security Context Constraints

AlphaPaaS applies the `alphapaas-restricted-v2` Security Context Constraint (SCC) automatically to all customer pods. This SCC is based on the OpenShift `restricted-v2` SCC and differs from it only by permitting Azure Disk CSI volume mounts.

CKS-01 All workloads MUST run under the `alphapaas-restricted-v2` SCC or a more restrictive SCC. Requests for elevated SCCs (e.g. `anyuid`, `privileged`) require documented justification and Group Security approval.

CKS-02 Containers MUST NOT run with `privileged: true`. Containers MUST NOT request the `hostNetwork`, `hostPID`, `hostIPC`, or `hostPath` capabilities.

CKS-03 Containers MUST set `allowPrivilegeEscalation: false` in their security context. Containers MUST drop all Linux capabilities (`drop: [ALL]`) and may add only `NET_BIND_SERVICE` if required, with justification.

CKS-04 Containers MUST NOT run as root (UID 0). The `alphapaas-restricted-v2` SCC enforces this by assigning a UID from a pre-allocated range; workloads MUST build images that honour this, with writable directories owned by GID 0.

CKS-05 Containers MUST set `securityContext.readOnlyRootFilesystem` explicitly. Workloads that require a writable root filesystem MUST document the reason; absence of the field (default) is itself a breach of this clause.

3. Image security

CKS-06	Container images MUST be pinned by digest (sha256) or by semantic version tag. The `:latest` tag MUST NOT be used in production or pre-production manifests. This applies to application images, base images, and BuildConfig ImageStreamTag references.
CKS-07	Container images MUST be pulled from approved registries: the AlphaPaaS integrated registry, AlphaInsure Artifactory, Amazon ECR, Azure Container Registry, or a signed-by-default public registry (Red Hat registry, Chainguard). Images from unsigned public registries (Docker Hub, unless a signed source is configured) are prohibited.
CKS-08	Container images MUST be scanned for vulnerabilities before deployment. Images with unremediated Critical or High severity vulnerabilities MUST NOT be deployed to production. The scan MUST be performed against a vulnerability database no older than 24 hours.
CKS-09	Image scan results MUST be recorded and made available to the Cloud Broker on request. Advanced Cluster Security (ACS) is the approved scanning solution; other tools may be used in addition but not as a replacement.

4. Secrets handling

Secret material includes passwords, API keys, TLS private keys, OAuth client secrets, database connection strings containing credentials, and any other data whose disclosure would grant unauthorised access.

CKS-10	Secrets MUST be sourced from an external secret manager: AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault. Secrets MUST be delivered to pods via the Secrets Store CSI driver or the External Secrets Operator. Kubernetes-native Secret resources may be used as intermediate storage but MUST be populated from an external manager, not committed to source control.
CKS-11	Secrets MUST be mounted into pods as file volumes, not exposed as environment variables. Environment variables are written to logs, system dumps, and process listings, creating unintended disclosure risk. Using `envFrom.secretRef` or `env.valueFrom.secretKeyRef` to inject secrets as environment variables is prohibited for new deployments.
CKS-12	Workloads consuming rotating secrets MUST detect rotation and either restart or reload without external intervention. Workloads that cannot tolerate rotation MUST document this limitation and include it in their runbook.
CKS-13	TLS private keys, including those used inside the cluster for mTLS, MUST be stored in a secret manager with rotation enabled. Long-lived TLS keys (validity > 12 months) are prohibited.

IMPORTANT — Do not commit secrets

Secrets committed to source control, even to private repositories, are considered disclosed. Rotation is mandatory on discovery, and the incident MUST be reported. Pre-commit hooks with secret scanning are strongly recommended.

5. Role-based access control

CKS-14	Roles and ClusterRoles MUST NOT use the wildcard verb or resource (*). Permissions MUST be specified explicitly. This aligns with CIS 5.1.3.

CKS-15	Permissions to read secrets at the namespace level MUST be granted only to ServiceAccounts that demonstrably require them. Cluster-wide read access to secrets is prohibited for application ServiceAccounts. This aligns with CIS 5.1.2.
CKS-16	Permissions to create pods MUST be granted only to controllers and CI/CD systems, not to human users or application ServiceAccounts. This aligns with CIS 5.1.4.
CKS-17	Role bindings MUST target specific ServiceAccounts or groups, never the `system:authenticated` or `system:unauthenticated` groups.

6. Network policies

CKS-18	Namespaces MUST have NetworkPolicies that restrict both ingress and egress traffic to the minimum required for the workload. This aligns with CIS 5.3.2. Detailed requirements for NetworkPolicy composition are defined in the Application Hosting Standard (AHS-12 to AHS-15).

7. Service account tokens

CKS-19	Workloads that do not interact with the Kubernetes API MUST set `automountServiceAccountToken: false`. Workloads that do interact with the API MUST use projected volume tokens with audience and expiration set, not long-lived ServiceAccount tokens mounted via the legacy secret-based mechanism. This aligns with CIS 5.1.6.
CKS-20	Long-term ServiceAccount tokens stored as Secrets MUST NOT be referenced by pods. Where legacy workloads mount such tokens via a Secret volume, the workload MUST be migrated to projected token volumes by the next release.

8. Runtime security monitoring

CKS-21	All production workloads run under Advanced Cluster Security (ACS) monitoring. Teams MUST triage ACS policy violations raised against their namespace within five business days. Critical severity violations MUST be remediated within 24 hours or an exception requested.

9. Related standards

- **AHS** — AlphaPaaS Application Hosting Standard
- **ARS** — AlphaPaaS Availability & Resilience Standard
- **ASC** — AlphaPaaS Approved Services Catalog
- **DCH** — AlphaPaaS Data Classification & Handling Standard